Vignette: Basic Implementation Check (Permutation Test on ECG Age Gap)

1. Overview

This package implements the core workflow for our project:

a **two-sample permutation test** comparing participants with atrial fibrillation (AF) versus those without AF on an ECG-derived continuous outcome called *Age Gap*:

$$g = \texttt{nn\_predicted\_age} - \texttt{age}.$$

The goal is to test whether the distribution of g differs between AF and non-AF groups using a nonparametric permutation procedure.

2. Functions Implemented

The basic pipeline uses four key functions:

1. `load_ecg_data(path)`

   Reads the dataset metadata file and keeps the needed columns:

   `patient_id`, `exam_id`, `age`, `nn_predicted_age`, `AF`.

2. `filter_to_unique_patient(df, method = "latest" | "random", seed = ...)`

   Keeps **one exam per patient** to reduce repeated-measures dependence.

3. `compute_age_gap(df)`

   Adds the variable

   $$\texttt{age\_gap} = \texttt{nn\_predicted\_age} - \texttt{age}.$$

4. `run_permutation_test(x, y, stat = "mean" | "median" | "ks", B, seed)`

   Runs the permutation test and returns `t_obs`, `t_perm`, `p_value`, and related metadata.

3. Demonstration with Synthetic Data

For the vignette, we use a small synthetic dataset so that anyone can run the example without needing the real CODE-15% files.

```r
set.seed(1)

n_patients <- 200
exams_per_patient <- sample(1:3, n_patients, replace = TRUE)
N <- sum(exams_per_patient)

df <- data.frame(
  patient_id = rep(seq_len(n_patients), times = exams_per_patient),
```

```
  exam_id = sample(100000:5000000, N, replace = FALSE),
  age = sample(18:95, N, replace = TRUE),
  nn_predicted_age = round(rnorm(N, mean = 55, sd = 12), 5),
  AF = sample(c(TRUE, FALSE), N, replace = TRUE, prob = c(0.15, 0.85))
)

head(df)
#>   patient_id exam_id age nn_predicted_age    AF
#> 1          1 4787395  27         50.38434 FALSE
#> 2          2 3263944  30         61.17839 FALSE
#> 3          2 1981044  54         52.15138  TRUE
#> 4          2 4232992  68         75.89813 FALSE
#> 5          3 3548531  83         69.45013 FALSE
#> 6          4 3524725  28         62.87170 FALSE
```

3.1 Keep One Exam per Patient

Because some patients have multiple exams, we demonstrate both supported methods:

- `"latest"`: keep the largest `exam_id` per patient
- `"random"`: randomly select one exam per patient (reproducible by seed)

```
df_latest <- filter_to_unique_patient(df, method = "latest")
df_random <- filter_to_unique_patient(df, method = "random", seed = 99)

nrow(df_latest)
#> [1] 200
length(unique(df$patient_id))
#> [1] 200
```

3.2 Compute Age Gap

```
df_latest <- compute_age_gap(df_latest)
summary(df_latest$age_gap)
#>    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#> -65.774 -20.110  -2.606  -1.337  17.762  55.030
```

3.3 Run Permutation Test

We form two samples: - Group 1: AF == TRUE - Group 2: AF == FALSE

Then run permutation test with a selectable statistic.

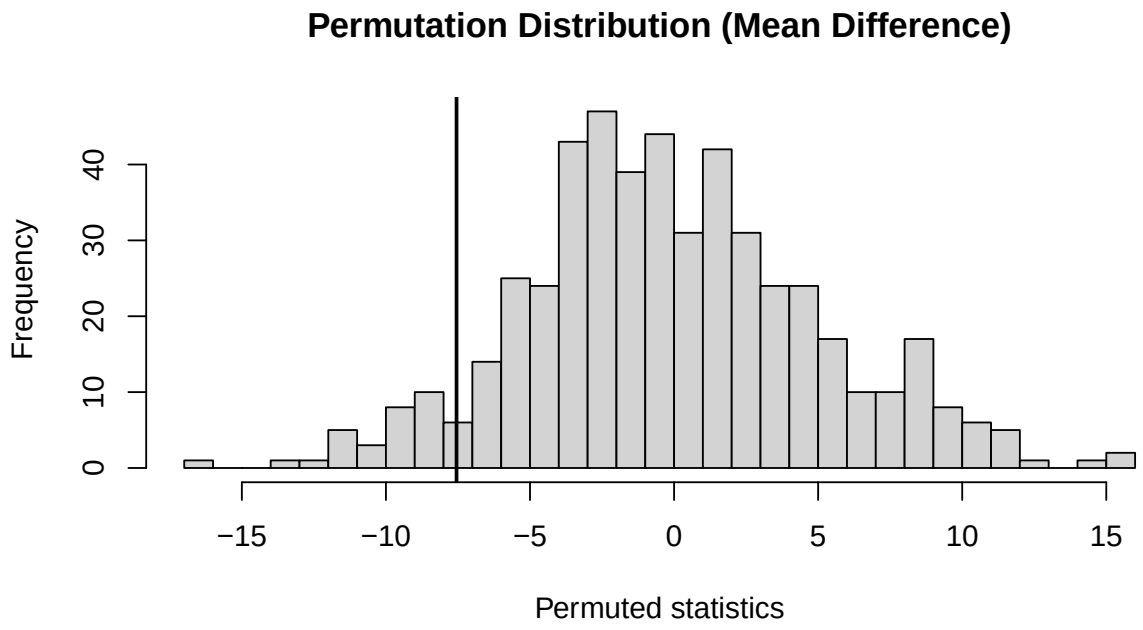
```
x <- df_latest$age_gap[df_latest$AF == TRUE]
y <- df_latest$age_gap[df_latest$AF == FALSE]

res_mean <- run_permutation_test(x, y, stat = "mean", B = 500, seed = 123)
res_median <- run_permutation_test(x, y, stat = "median", B = 500, seed = 123)
res_ks <- run_permutation_test(x, y, stat = "ks", B = 500, seed = 123)

res_mean$p_value
#> [1] 0.1457086
res_median$p_value
#> [1] 0.3373253
```

```
res_ks$p_value
#> [1] 0.2914172
```

4. Graphs: permutation distribution

A required output is the permutation distribution plot with a vertical line at the observed statistic.

```
hist(res_mean$t_perm, breaks = 30,
     main = "Permutation Distribution (Mean Difference)",
     xlab = "Permuted statistics")
abline(v = res_mean$t_obs, lwd = 2)
```



5. Brief Assumption / Caveat Check

Permutation testing assumes that under H_0, the group labels are exchangeable.

A practical issue in this dataset is repeated exams per patient, which violates strict independence.

Our default approach is to keep only one exam per patient using `filter_to_unique_patient()`.

(An optional extension in the full project is patient-level block permutation.)

6. Testing (testthat) — Sample Outputs Shown

This checkpoint requires writing tests with **testthat** and showing some outputs in the vignette.

Below is a short snippet of what our tests validate:

- `compute_age_gap()` correctly creates

$$\texttt{age\_gap} = \texttt{nn\_predicted\_age} - \texttt{age}.$$

- `filter_to_unique_patient()` returns one row per patient (latest method)

- `filter_to_unique_patient(..., method = "random")` is reproducible with the same seed
- `run_permutation_test()` returns the correct structure and valid p-values
- An invalid `stat` argument triggers an error

```
if (requireNamespace("testthat", quietly = TRUE)) {
  testthat::test_file("test_code.R", reporter = "summary")
} else {
  cat("Package 'testthat' is not installed in this environment.\n",
      "Install it to run and display the full test output from test_code.R.\n")
}
#> code: ..............
#>
#> == DONE ========================================================================
```

7. References (BibTeX)

Ribeiro, A. H., M. H. Ribeiro, G. M. M. Paixao, et al. 2021. "CODE-15%: A Large-Scale Electrocardiogram Dataset for Deep Learning." Zenodo. https://doi.org/10.5281/zenodo.4916206.